# Competitive Programming Handsheet

This document contains essential code templates and algorithms for competitive programming contests.

## 1 Graph Algorithms

### BFS and DFS(Graph Traversal Algorithms)

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef vector<ll> vi;
typedef vector<vi> vvi;

/* ================ actual code starts here ================ */

void bfs(const vvi& adj, ll start) {
    vector<bool> visited(adj.size(), false);
    queue<ll> q;
    q.push(start);
    visited[start] = true;

    while (!q.empty()) {
        ll node = q.front();
        q.pop();
        cout << node << " ";

        for (ll neighbor : adj[node]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }
}

void dfs(const vvi& adj, ll node, vector<bool>& visited) {
    if (visited[node]) return;

    visited[node] = true;
    cout << node << " ";

    for (ll neighbor : adj[node]) {
        dfs(adj, neighbor, visited);
    }
}

void bfsMatrix(const vvi& adj, ll start) {
    ll n = adj.size();
    vector<bool> visited(n, false);
    queue<ll> q;

    q.push(start);
    visited[start] = true;

    while (!q.empty()) {
        ll node = q.front();
        q.pop();
        cout << node << " ";

        for (ll i = 0; i < n; i++) {
            if (adj[node][i] == 1 && !visited[i]) {
                visited[i] = true;
                q.push(i);
            }
        }
    }
}
```

```cpp

void dfsMatrix(const vvi& adj, ll node, vector<bool>& visited) {
    if (visited[node]) return;

    visited[node] = true;
    cout << node << " ";

    for (ll i = 0; i < adj.size(); i++) {
        if (adj[node][i] == 1) {
            dfsMatrix(adj, i, visited);
        }
    }
}
```

Listing 1: BFS and DFS(Graph Traversal Algorithms)

## Prim's Algorithm (Minimum Spanning Tree)

```cpp
#include<bits/stdc++.h>
using namespace std;

// Function to find sum of weights of edges of the Minimum Spanning Tree.
int spanningTree(int V, int E, vector<vector<int>> &edges) {

    // Create an adjacency list representation of the graph
    vector<vector<int>> adj[V];

    // Fill the adjacency list with edges and their weights
    for (int i = 0; i < E; i++) {
        int u = edges[i][0];
        int v = edges[i][1];
        int wt = edges[i][2];
        adj[u].push_back({v, wt});
        adj[v].push_back({u, wt});
    }

    // Create a priority queue to store edges with their weights
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;

    // Create a visited array to keep track of visited vertices
    vector<bool> visited(V, false);

    // Variable to store the result (sum of edge weights)
    int res = 0;

    // Start with vertex 0
    pq.push({0, 0});

    // Perform Prim's algorithm to find the Minimum Spanning Tree
    while(!pq.empty()){
        auto p = pq.top();
        pq.pop();

        int wt = p.first;  // Weight of the edge
        int u = p.second;  // Vertex connected to the edge

        if(visited[u] == true){
            continue;  // Skip if the vertex is already visited
        }

        res += wt;  // Add the edge weight to the result
        visited[u] = true;  // Mark the vertex as visited

        // Explore the adjacent vertices
        for(auto v : adj[u]){
            // v[0] represents the vertex and v[1] represents the edge weight
            if(visited[v[0]] == false){
                pq.push({v[1], v[0]});  // Add the adjacent edge to the priority queue
            }
```

```
        }
    }

    return res;  // Return the sum of edge weights of the Minimum Spanning Tree
}

int main() {
    vector<vector<int>> graph = {{0, 1, 5},
                                 {1, 2, 3},
                                 {0, 2, 1}};

    cout << spanningTree(3, 3, graph) << endl;

    return 0;
}
```

Listing 2: Prim's Algorithm (Minimum Spanning Tree)

## Bellman-Ford Algorithm (Shortest Path with Negative Edges)

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef vector<ll> vi;
typedef vector<vi> vvi;

/* ================ actual code starts here ================ */

ll bellmanFord(int V, vvi& edges, vi& dist, ll start) {
    dist[start] = 0;
    ll u, v, w;
    for (int i = 0; i < V; i++) {
        for (auto edge : edges) {
            u = edge[0];
            v = edge[1];
            w = edge[2];

            if (dist[u] != LLONG_MAX && dist[u] + w < dist[v]) {
                if (i == V - 1) return -1;
                dist[v] = w + dist[u];
            }
        }
    }
    return 0;
}

void solve(ll tc) {
    ll n, e;
    cin >> n >> e;

    vvi edges;

    ll u, v, w;
    while (e-- > 0) {
        cin >> u >> v >> w;
        edges.push_back({ u, v, w });
    }

    ll start = 1;
    vi dist(n, LLONG_MAX);

    ll f = bellmanFord(n, edges, dist, 0);

    if (f != -1) {
        for (auto i : dist) cout << i << ' ';
    }
    else cout << -1 << endl;
}
```

Listing 3: Bellman-Ford Algorithm (Shortest Path with Negative Edges)

## Floyd-Warshall Algorithm (All-Pairs Shortest Path)

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = LLONG_MAX / 4;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<ll>> dist(n, vector<ll>(n, INF));

    for (int i = 0; i < n; i++)
        dist[i][i] = 0;

    for (int i = 0; i < m; i++) {
        int u, v;
        ll w;
        cin >> u >> v >> w;
        dist[u][v] = min(dist[u][v], w);
    }

    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            if (dist[i][k] == INF) continue;
            ll dik = dist[i][k];
            for (int j = 0; j < n; j++) {
                if (dist[k][j] == INF) continue;
                ll nd = dik + dist[k][j];
                if (nd < dist[i][j])
                    dist[i][j] = nd;
            }
        }
    }

    bool hasNegCycle = false;
    for (int i = 0; i < n; i++) {
        if (dist[i][i] < 0) {
            hasNegCycle = true;
            break;
        }
    }

    if (hasNegCycle) {
        cout << "Graph contains a negative-weight cycle\n";
    } else {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][j] == INF)
                    cout << "INF ";
                else
                    cout << dist[i][j] << ' ';
            }
            cout << '\n';
        }
    }

    return 0;
}
```

Listing 4: Floyd-Warshall Algorithm (All-Pairs Shortest Path)

# 2 Number Theory

## Binary Exponentiation

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

const int MOD = 1e9 + 7;
ll binpow(ll a, ll b) {
  ll res = 1;
  while (b > 0) {
    if (b & 1)
      res = (res * a) % MOD;
    a = (a * a) % MOD;
    b >>= 1;
  }
  return res;
}

//================actual code starts from here=============================================
void solve() {
  cout << binpow(100, 3);
}

int main() {
  ios_base::sync_with_stdio(false);
  cin.tie(NULL);
  cout.tie(NULL);

  ll t = 1;
  cin >> t;
  while (t--)
    solve();

  return 0;
}
```

Listing 5: Binary Exponentiation

## Sieve of Eratosthenes

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef vector<ll> vi;

const ll N = 10000003;
bitset <N> mark;
vi primes;

void sieve_of_eratosthenes() {
  primes.push_back(2);
  mark[0] = mark[1] = 1;
  int lim = sqrt(N * 1.0) + 2;

  for (int i = 4; i < N; i += 2) mark[i] = 1;
  for (int i = 3; i < N; i += 2) {
    if (!mark[i]) {
      primes.push_back(i);
      if (i <= lim) {
        for (int j = i * i; j < N; j += i) {
          mark[j] = 1;
        }
      }
    }
```

```
  }
}
```

Listing 6: Sieve of Eratosthenes

# 3 Bitwise

## Bitwise Cheat Sheet

```cpp
#include <iostream>
using namespace std; // Using the entire std namespace

int main() {
  int x = 42; // Example value
  int i = 3;  // Example bit position
  int j = 1;  // Another example bit position
  // Arithmetic operations
  cout << "1 << i (pow(2, i)): " << (1 << i) << endl;
  cout << "x << i (x * pow(2, i)): " << (x << i) << endl;
  cout << "x >> i (floor(x / pow(2, i))): " << (x >> i) << endl;
  cout << "x & ((1 << i) - 1) (x % pow(2, i)): " << (x & ((1 << i) - 1)) << endl;
  // Bitmask construction
  cout << "(1 << i) (bitmask with only i-th bit on): " << (1 << i) << endl;
  cout << "(1 << i) | (1 << j) (bitmask with only i-th and j-th bits on): " << ((1 << i) |
    (1 << j)) << endl;
  cout << "~0 (bitmask with all bits on): " << (~0) << endl;
  cout << "~(1 << i) (bitmask with all bits on except i-th bit off): " << (~(1 << i)) <<
    endl;
  cout << "-1 ^ (1 << i) (same as above): " << (-1 ^ (1 << i)) << endl;
  // Bit operations on variable x
  cout << "Original x: " << x << endl;
  cout << "x | (1 << i) (turn on the i-th bit of x): " << (x | (1 << i)) << endl;
  cout << "x & ~(1 << i) (turn off the i-th bit of x): " << (x & ~(1 << i)) << endl;
  cout << "x ^ (1 << i) (toggle (flip) the i-th bit of x): " << (x ^ (1 << i)) << endl;
  cout << "(x >> i) & 1 (check if the i-th bit of x is on): " << ((x >> i) & 1) << endl;
  // Tricks and built-in functions
  long long x_ll = 10; // Example for __builtin_popcountll
  cout << "x & -x (isolate the lowest bit that is on (LSB)): " << (x & -x) << endl;
  cout << "__builtin_popcountll(" << x_ll << ") (count number of bits that are on): " <<
    __builtin_popcountll(x_ll) << endl;
  return 0;
}
```

Listing 7: Bitwise Cheat Sheet